

Módulo: [BE-JV-010] Arquitetura de Software e Ágil II

Prazo de Entrega: 15 de Julho

Formato: Grupos de, no mínimo, 4 pessoas.

🚀 **Objetivo do Projeto**

Desenvolver um ecossistema distribuído de microsserviços responsável por processar pagamentos de faturas via PIX, emitir comprovantes e notificar os clientes. O projeto deve aplicar os conceitos de Domain-Driven Design (DDD), comunicação assíncrona, cache, resiliência e testes de contrato.

🏗️ **Requisitos Arquiteturais (O que deve ser construído)**

A solução deverá ser dividida em microsserviços com bancos de dados segregados. A arquitetura mínima exigida contempla:

1. Microsserviço de Pagamento/Fatura (Orquestrador SAGA)

- Recebe a requisição de pagamento via PIX.
- Garante a consistência dos dados utilizando o **padrão SAGA** para assegurar que a fatura seja dada como paga apenas se o comprovante for gerado com sucesso.

2. Microsserviço de Comprovantes (Filas e Cache)

- **Inserção (POST):** Deve receber a requisição e validar as informações obrigatórias. Estando tudo correto, a API retorna um 202 - Accepted com o ID gerado (UUID v4) e posta a mensagem em uma **fila (RabbitMQ)**. Um *consumer* deverá ler essa fila e gravar o comprovante no banco de dados.
- **Consulta (GET):** Ao buscar um comprovante pelo ID, a busca deve ocorrer **primeiramente no cache compartilhado (Redis)**. Se não encontrar, busca no banco, salva no Redis e retorna ao usuário. Caso não encontre após 3 re-tentativas, deve retornar 404 - Not Found.

3. Microsserviço de Notificação do Cliente (Tópicos e Resiliência)

- Após a gravação do comprovante, um evento de "Pagamento Realizado com Sucesso" deve ser publicado em um **tópico do Kafka**.
- O microsserviço de cliente atuará como *subscriber*, consumindo essa mensagem para "notificar" o usuário.
- **Obrigatório:** Implementar mecanismos de *retry* utilizando a anotação `@RetryableTopic`.

4. Testes de Contrato

- Utilizar o framework **PACT** para garantir a integridade da comunicação (Contract Testing) entre os microserviços (ex: entre o serviço de Pagamento e o serviço de Comprovantes).

Especificação de Contratos (Payloads)

O *Request Body* para a geração do comprovante deve seguir o formato:

```
{
  "nome": "Giovanni Vicente",
  "tipo_documento": "CPF",
  "numero_documento": "50329291076",
  "numero_agencia": "2022",
  "numero_conta": "00276",
  "digito_verificador_conta": "0",
  "valor_transacao": 23.99,
  "tipo_chave_pix_destino": "CELULAR",
  "chave_pix_destino": "11948755536",
  "nome_cliente_destino": "Fernando Augusto",
  "identificacao_pix": "Segue pagamento da minha cota no churrasco de domingo",
  "data_hora_transacao": "2022-04-10T20:03:57.116061100"
}
```

A resposta de sucesso (202 - Accepted) deve retornar:

```
{
  "identificador_comprovante": "b819cc65-f6f0-478c-8bb7-69ee1c4f6402",
  "data_hora_requisicao": "2022-04-10T20:03:57.116061100"
}
```

Sugestão de Divisão de Tarefas (Para grupos de 4+)

Para garantir que todos codifiquem e participem ativamente, sugere-se a seguinte divisão de responsabilidades, embora todos devam compreender o ecossistema completo:

- **Membro 1 (Core & SAGA):** Criação do Microserviço de Pagamentos/Faturas, configuração do banco de dados (H2 ou relacional à escolha) e implementação da orquestração SAGA.
- **Membro 2 (Mensageria e Persistência):** Construção da rota POST de comprovantes, configuração do *Producer* e *Consumer* no **RabbitMQ** para gravação assíncrona no banco.
- **Membro 3 (Performance e Leitura):** Implementação da rota GET de comprovantes, integração com **Redis** para a estratégia de *Cache*, incluindo as lógicas de *cache miss* e as 3 re-tentativas antes do 404.
- **Membro 4 (Eventos, Resiliência e Qualidade):** Configuração do **Kafka**, publicação e consumo do tópico de notificações, implementação do `@RetryableTopic` e criação dos Testes de Contrato (PACT).
- *Membros adicionais (se houver):* Podem atuar no auxílio da documentação de Replicação em Cloud, ampliação da cobertura de testes ou criação de um *API Gateway*.

Critérios de Avaliação (Rubricas)

O grupo será avaliado de acordo com a apropriação e aplicação prática das seguintes competências:

1. **Arquitetura de Microsserviços:** Aplicação correta de DDD e separação de domínios.
2. **Sessões e Cache:** Uso eficiente do cache compartilhado com Redis.
3. **Comunicação Assíncrona (Filas e Tópicos):** Domínio prático dos conceitos de *producer* e *consumer* usando RabbitMQ e Kafka.
4. **Arquitetura Amigável a Testes:** Sucesso na implementação dos Testes de Contrato.
5. **Cloud Replicado:** Como não há base legado para migrar, o grupo deve entregar um breve documento (ou arquivo README) contendo uma pesquisa teórica detalhando a estratégia de replicação em Cloud para a aplicação construída